

Data Mining

Block 1 - What it is, and how not to fool yourself

Daniel Wlazło

SGH Warsaw School of Economics - SMMD-ADA

06 October 2026

About me

- Data Science manager & data scientist - specialising in **credit risk** and **fraud detection**
- PhD student at SGH
- MSc in Computer Science and in Automatic Control & Robotics, Warsaw University of Technology
- Homepage: `datadeer.pl`

Expect many examples from retail credit risk - not because it is the course domain, but because it is the field closest to me.

Today, hour by hour

- ① **Course logistics** - rules, dates, grading ~10 min
- ② **What data mining is** - the map of learning problems, CRISP-DM ~35 min
- ③ **EDA** - how to actually read a dataset ~50 min
- ④ **Missingness & feature engineering** - today's core skills ~35 min
- ⑤ **The craft** - habits that save beginner data scientists ~20 min
- ⑥ **Lab** - notebook 01_eda_missingness ~30 min

Breaks roughly every 50 minutes. Questions welcome at any point.

The deal, in one slide

Seven Tuesdays, 17:10–20:30

- 6 teaching blocks + 1 for defences & exam
- Language: English; code in Python
- Running examples: **retail credit risk**

Passing

- Project **50%** + Exam (term 0) **50%**
- Pass with $\geq 60\%$ overall
- Project defended in Block 7; exam same day

Where we are going

#	Topic	Date
1	Data mining, CRISP-DM, EDA & missingness	06.10
2	Linear & logistic regression	13.10
3	Trees, ensembles (RF, GBDT), honest evaluation	20.10
4	Clustering & dimensionality reduction	27.10
5	Pattern discovery (Market Basket) & text mining	03.11
6	Neural networks & responsible ML	10.11
7	Project defences + exam (term 0)	17.11

One running example throughout: modelling a credit portfolio well.

The project: what exactly you will build

- In a **team of 3**: a full mini CRISP-DM cycle on a dataset of **your choice** - business framing → data preparation → **at least two model families** → correct evaluation → interpretation → a recommendation.
- Data: any real dataset that can carry the cycle - a meaningful business decision, a predictable target, enough rows for an honest split (Kaggle, UCI, open data...). Declared and approved via the topic description; unsure? ask early.
- The project **ends with a presentation of results** in Block 7 (the last class), ~6–8 minutes + questions.

Deliverables - both uploaded **before** Block 7 starts

(i) all code, as a reproducible notebook; (ii) the exact presentation you will show. What you upload is what you present.

The project: timeline & deadlines

Deadline	What is due
before Block 2 (13.10)	teams of 3 formed and registered
before Block 3 (20.10)	topic chosen + a short written description (the decision, the dataset, the target, the planned two model families)
before Block 7 (17.11)	upload all code + the final presentation
Block 7 (17.11)	presentation of results + Q&A

No topic idea? **Consult before the deadline** - e-mail or after class. That is what the consultations are for.

The project: a sensible week-by-week split

Week	Focus - with the suggested methods
1	find two teammates; first contact with a candidate dataset
2	topic + description; EDA: distributions, target, missingness
3	data prep (impute + indicators, one-hot); baseline : logistic regression in a pipeline, honest split; leakage audit
4	second family: random forest / GBDT; evaluation kit: CV, ROC/PR, calibration, cost threshold
5	optional: clustering + PCA, rules, or TF-IDF; interpretation → recommendation; presentation skeleton
6	freeze notebook (Restart & Run All); slides; rehearse; upload
7	present the results

Each week's block teaches exactly what that week's project step needs - stay on this schedule and the project builds itself alongside the course.

The project: grading & the AI rule

Rubric (50% of the course)

- framing & problem definition - 10%
- data prep & missingness - 15%
- modelling: ≥ 2 families - 15%
- evaluation, leakage-aware - 20%
- interpretation & recommendation - 15%
- reproducibility - 5%
- presentation & defence - 20%

Use of AI

- allowed - but **you are responsible for every line you submit**
- the more the project feels produced *solely* by AI, the **harder the defence questions**
- expect to explain and modify your code *live*
- submissions are **read for fingerprints of unchecked generation** - conventions nobody in the team can explain count against you

Note the weights: the **presentation is a fifth of the grade** - a correct analysis you cannot communicate and defend is worth less than you think. Raw accuracy alone buys almost nothing.

Prerequisites & tooling

What we will work with

- **Python** - all labs and the project are in Python
- **Jupyter notebooks** - run locally or on Google Colab
- Worth refreshing before next week: **pandas** (DataFrames, filtering, groupby, joins)

Setting up your environment

- Recommended: `uv` - one tool for Python versions, virtual environments, and packages
- `uv sync` in the course repo gives you a ready-to-run environment

Rusty?

Notebook `00_python_primer` is your self-paced warm-up.

What data mining *is* - and is *not*

It is

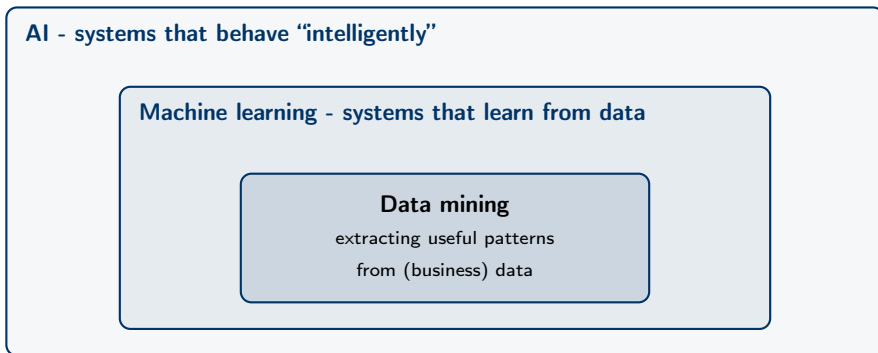
- finding useful structure in data
- to support a *decision*
- with methods that are honest about uncertainty

It is not

- running every algorithm and keeping the highest accuracy
- “the model said so”
- a substitute for understanding the business problem

We start from the **decision**, then choose the method - never the reverse.

Data mining, statistics, ML, AI - who owns what?



- **Statistics** underpins all three rings: inference, uncertainty, experimental design.
- In practice the borders are soft - employers say "data science" and mean the union.

Three ways a machine can learn

Supervised

- data comes with **labels**
- learn to predict the label for new cases
- *will this customer default?*

Unsupervised

- no labels - find **structure**
- clusters, patterns, compressed representations
- *which customers are similar?*

Reinforcement

- an agent acts, gets **rewards**, adapts
- *which collections action to try next?*

This course: **supervised** (Blocks 2–3, 6) and **unsupervised** (Blocks 4–5). Reinforcement learning we only mention - so you know where it sits on the map.

Supervised learning comes in flavours

Task	Target	Credit example
Binary classification	two classes	<code>default</code> $\in \{0, 1\}$
Multiclass classification	several classes	complaint category; rating grade
Regression	a number	loss amount; recovery rate
<i>... and more</i>	ordinal, ranking, survival time	months until default

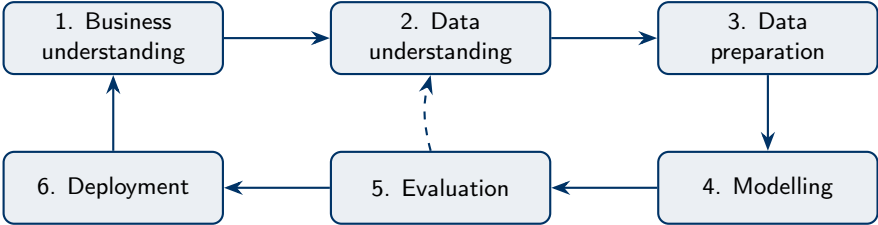
Our focus: **binary classification** and **regression**.
 The rest we will meet in passing - enough to recognise them in the wild.

From business question to learning task

Business question	Learning task	Covered in
Will this applicant default?	binary classification	Blocks 2–3, 6
How large will the loss be if they do?	regression	Blocks 2–3
Which complaint team should read this email?	multiclass classification	Block 5 (mention)
Do our customers form natural segments?	clustering	Block 4
Which products are bought together?	association rules	Block 5

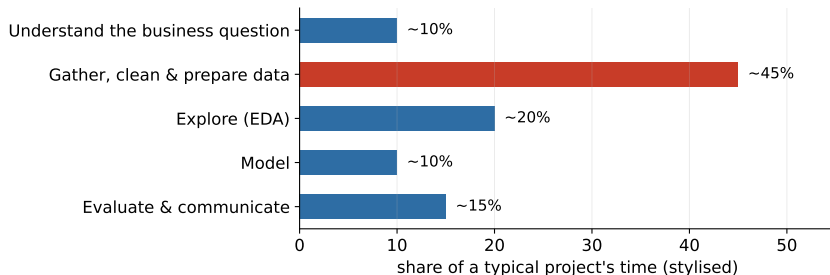
The translation step is **your** job - no algorithm does it for you.
Getting it wrong here invalidates everything downstream.

The spine of the course: CRISP-DM



Today lives in boxes **1 and 2**. The dashed arrow is the point: evaluation sends you back to understanding.

Where a project's time actually goes



- Modelling - the part courses spend most time on - is a **small slice** of real work.
- That is why this course starts with data understanding, not with algorithms.

Our business problem for the whole term

The lender's question

Given what we know about a customer at application time, how likely are they to **default**? And what should we *do* about it?

- Each row = one credit customer; target `default` $\in \{0, 1\}$.
- Realistic ingredients: income, debt-to-income, utilisation, prior delinquency, existing loans, savings, purpose. . .
- Realistic problems: **class imbalance**, **missing data**, and **leakage** - all of which we meet today or next week.

Today: the synthetic portfolio via `finance_data.load_credit()`; from Block 2, the real Taiwan portfolio via `load_taiwan()` - same lender's question.

First decision nobody teaches: what is a *row*?

The unit of analysis - what one row of your table represents:

- one **application**? one **customer**? one **customer-month**? one **transaction**?

Why it matters

- The unit fixes what the prediction *means* - and when it can be used.
- A customer with 3 loans: three rows or one? Your metrics change either way.
- Mixing units (some rows customers, some applications) silently **double-counts** people.

In our portfolio: one row = one **customer with one loan application**.

Meet the dataset

Column (sample)	Type	Notes
customer_id	identifier	never a feature!
age, monthly_income_pln	numeric	income has missing values
housing, purpose, region	categorical	no natural order
checking_status	categorical (ordinal)	none < low < medium < high
credit_utilization, debt_to_income	numeric ratios	key risk drivers
months_since_last_delinquency	numeric	structurally missing
collections_contact	0/1	a trap - more in Block 3
default	0/1	the target

4,000 customers, 18 columns; German Credit (OpenML) or a synthetic Polish portfolio - same interface.

The data you will never see: selection bias

Ask of every dataset: how did rows get *into* this table?

A lender's portfolio contains only **approved** loans. Rejected applicants have no default outcome - they are simply absent.

- Our model learns on the *survivors of yesterday's policy* - a form of survivorship bias.
- For applicants unlike anyone previously approved, the model **extrapolates** - its confidence there is borrowed, not earned.
- Credit scoring has a name and a toolbox for this (*reject inference*); most other domains quietly ignore it.
- Same pattern everywhere: churn models see only acquired customers, hiring models see only past hires.

Selection bias cannot be fixed by more data **from the same source**.

Before any model: look at the data

Data understanding checklist

- 1 Shape, columns, data types.
- 2 Target balance - how (im)balanced is default?
- 3 Univariate: distributions, implausible values, units.
- 4 Bivariate: which features move with the target?
- 5 **Data quality**: duplicates, outliers, and *missingness*.

Skipping this step is the most common way a data-mining project quietly fails.

First contact with a DataFrame

Four commands before anything else:

```
df.shape           # how much data do I have?
df.dtypes          # what does pandas think each column is?
df.head(10)        # do the values look like the docs promised?
df.describe(include="all") # ranges, counts, uniques, NaNs
```

What you are looking for

- numbers stored as strings ("12 000", "N/A", "-")
- categorical levels that differ only by case or spelling
- minima / maxima that make no physical sense
- a describe() count smaller than len(df) - that is missingness

Duplicates and keys

Two questions for every table

- What *should* uniquely identify a row? (the key - here: `customer_id`)
- Does it? `df["customer_id"].is_unique`

Where duplicates come from

- repeated exports glued together; joins that multiplied rows; customers applying twice

Why it is dangerous

- duplicated rows are double-counted evidence - metrics look better/worse than reality
- a duplicate that lands in both train and test set **inflates every score** (a leakage cousin - Block 3)

Know your variable types

Type	Example	Watch out for
Numeric, continuous	income, DTI	skew, outliers, units
Numeric, discrete	# existing loans	often better treated as categories
Categorical, nominal	purpose, region	rare levels, typos, encodings
Categorical, ordinal	checking status	order matters - keep it
Dates & times	application date	time zones; enables leakage
Text	complaint narrative	needs its own pipeline (Block 5)
Identifiers	customer_id	never a feature

The type decides the plot, the summary statistic, *and* the encoding.

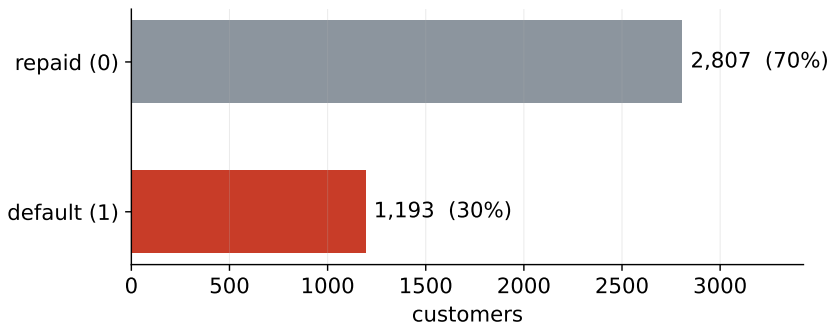
Scales of measurement (Stevens)

Scale	Defining property	Meaningful ops	Example
Nominal	categories, no order	$=, \neq$	purpose, region
Ordinal	ordered, gaps unknown	$<, >$	checking status
Interval	equal gaps, no true zero	$+, -$	temperature in °C; dates
Ratio	true zero exists	$\times, \div$	income, loan amount, age

Why it matters

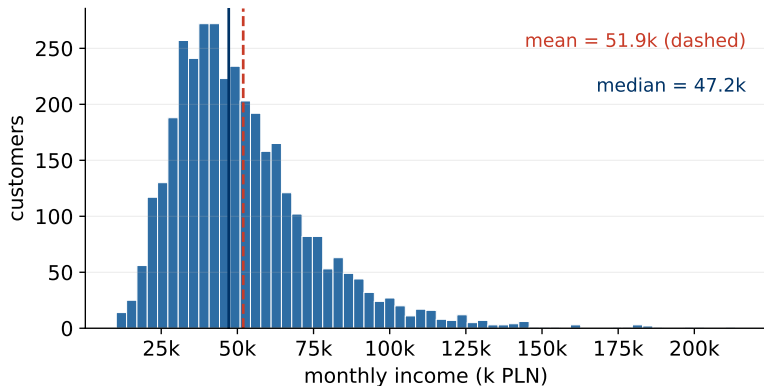
- The scale limits the statistics: a **mean of nominal codes is nonsense**; medians need at least ordinal.
- “Twice as much” needs a ratio scale: 40°C is not twice as warm as 20°C, but a 40k loan *is* twice a 20k loan.
- Encoding an ordinal variable as one-hot throws its order away; encoding a nominal one as integers invents an order that isn't there.

The target comes first: class balance



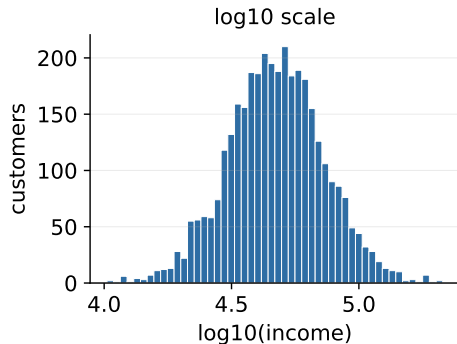
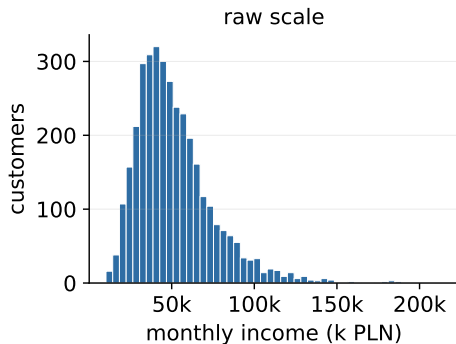
- A model that always says “repaid” is right 7 times out of 10 - and completely **useless**.
- Accuracy is therefore *not* the metric for imbalanced problems (proper treatment in Block 3).
- Real PD portfolios are far more extreme: 1–5% default rates.

Univariate EDA: the shape of a distribution



- Income is **right-skewed**: $\text{mean} > \text{median}$ - a few high earners pull the mean up.
- Skew is information: it tells you which summary to report and whether a transform will help.

Heavy tails? Change the lens



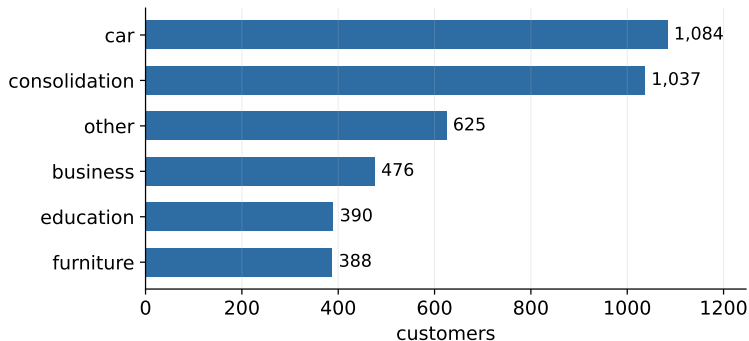
- On a log scale the same variable becomes nearly symmetric.
- Monetary amounts (income, savings, loss) almost always deserve a log look - **multiplicative** thinking fits money.

Sanity checks: implausible values

Red flag	Example	Likely cause
Impossible values	age = 150; income < 0	entry error, sentinel codes
Sentinel numbers	income = 999999	“missing” coded as a number
Unit mixes	salary 3,000 vs 36,000	monthly vs annual, PLN vs EUR
Too-round values	everyone earns exactly 5,000	defaults filled by a form
Frozen columns	one value everywhere	broken export, dead field

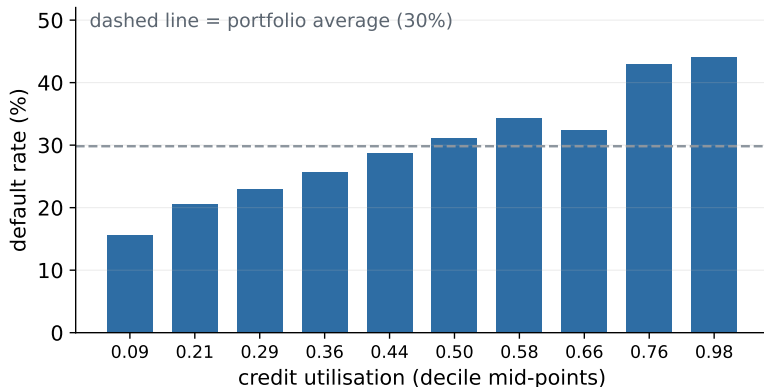
Every red flag is a **question for the data owner**, not a value to silently “fix”.

Categoricals: levels and rare categories



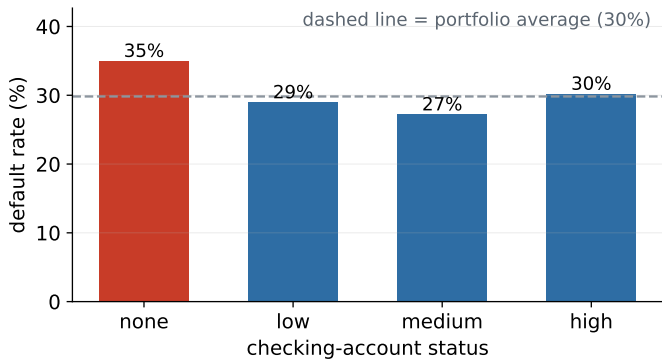
- Check levels: typos and case variants create phantom categories.
- Rare levels (<1–2% of rows) often get grouped into other - but only *after* checking they are not special (e.g. a tiny, very risky segment).

Bivariate EDA: numeric feature vs target



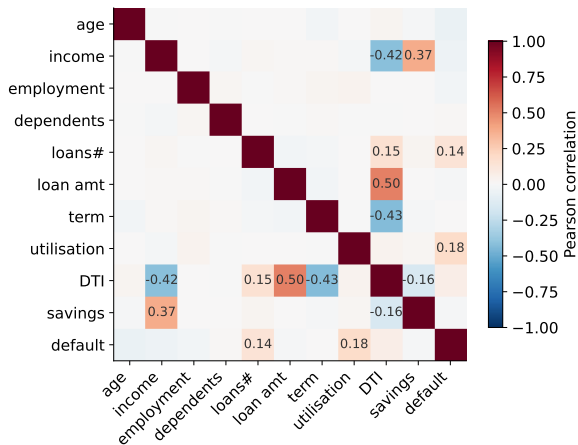
- Bin the feature, plot the default rate per bin - the **shape** of the risk relationship appears.
- Monotone and steep $\Rightarrow$ a strong candidate feature (utilisation is a classic credit driver).

Bivariate EDA: categorical feature vs target



- “No checking account” is the riskiest group - the bank cannot see those customers’ cash flows.
- Compare each level to the *portfolio average*, and mind small groups: a rate computed on 30 customers is noise.

A first map: the correlation matrix



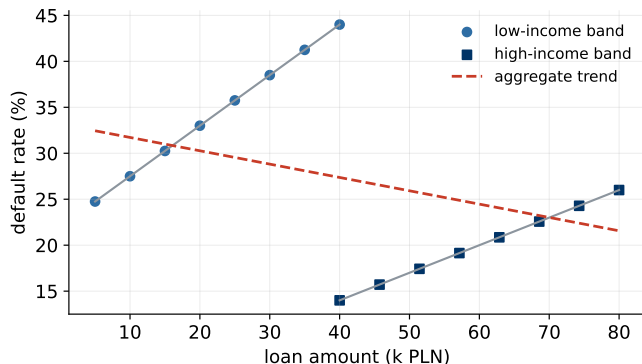
- One picture: which features move together, and which move with the target.
- Utilisation and the number of existing loans stand out for default - current debt load beats income here.
- Strongly correlated feature pairs ($|r| > 0.8$) are candidates for dropping one.

Correlation: handle with care

- **Correlation is not causation.** Ice cream sales correlate with drownings; summer causes both.
- **Pearson sees only straight lines.** A perfect U-shaped relationship can have $r \approx 0$.
- **Outliers can manufacture correlation.** One extreme point can turn $r = 0.1$ into $r = 0.7$.
- **Rank alternatives exist.** Spearman correlation asks only “do they move in the same direction?” - more robust for skewed data.

A correlation matrix generates **hypotheses**, not conclusions.

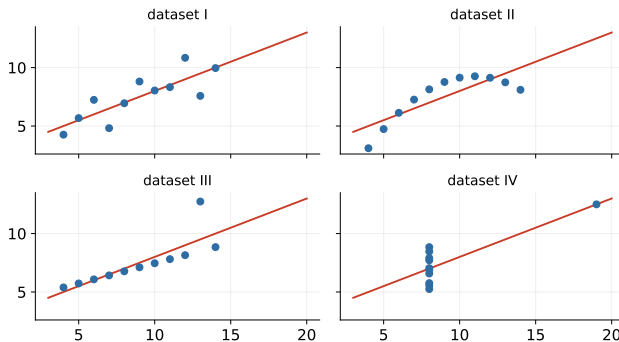
Simpson's paradox: the aggregate can lie



- In aggregate, bigger loans look *safer* - because high-income customers take the big loans.
- Within every income band, bigger loans are riskier. Both statements are true; only one is useful.
- Before concluding from any bivariate plot, ask: **compared within what?** (Illustrative data.)

Why you must plot: Anscombe's quartet

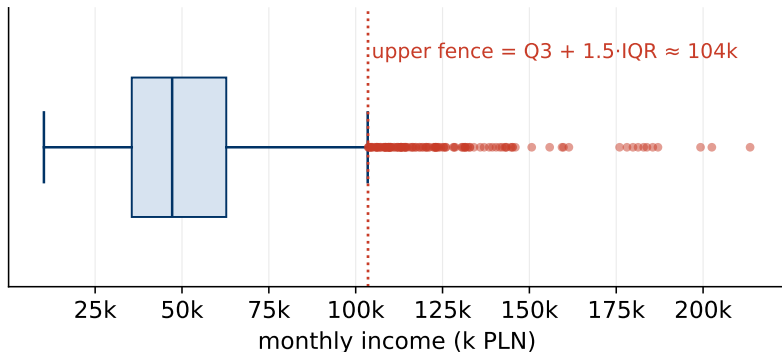
Four datasets, identical summaries: mean, variance, corr = 0.816, same regression line



Same mean, same variance, same correlation, same regression line - four completely different stories.

Summary statistics hide; plots reveal.

Outliers: how to spot them



- The boxplot's **IQR rule**: flag points beyond $Q3 + 1.5 \cdot IQR$ (or below $Q1 - 1.5 \cdot IQR$).
- A convention, not a law - for skewed variables it flags many perfectly genuine values (log first, then look again).

Outliers: what to actually do

- 1 **Investigate first.** Error or real? A 2 mln PLN income might be a typo - or a real customer who matters.
- 2 **If it is an error:** fix from source, or set to missing (do not invent a value).
- 3 **If it is genuine:** keep it; consider capping / winsorising or a log transform so it stops dominating.
- 4 **Document the rule** - “we capped income at the 99th percentile” must be written down and applied identically to new data.

Fraud-detection perspective

In fraud, outliers are not noise - they are often *the signal*. Whether an outlier is a problem depends on the business question.

EDA is a loop, not a phase

The rhythm

- ① ask a question
- ② make the one plot that answers it
- ③ the answer raises two new questions
- ④ repeat until surprises stop

Keep an evidence log

- every finding: one sentence + the plot
- “income is MNAR-missing for high earners”
beats 40 unlabelled histograms
- your future self (and your team) will re-read it

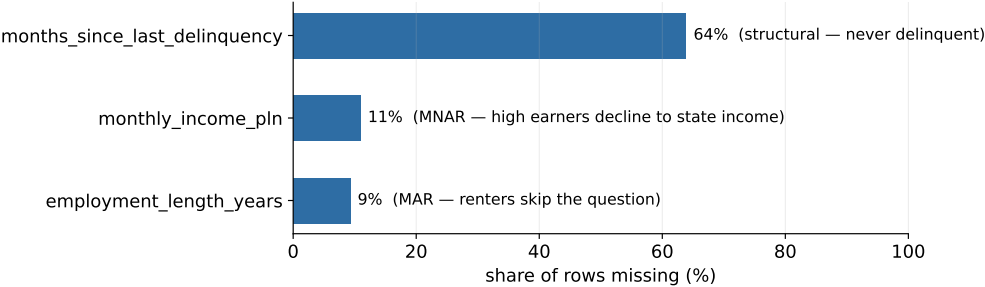
EDA never really ends - evaluation (CRISP-DM's dashed arrow) sends you back here.

Missing data is not one thing

Type	Meaning	Credit example
MCAR	missing completely at random	a system glitch drops a field
MAR	depends on <i>observed</i> data	renters skip employment length
MNAR	depends on the <i>unobserved</i> value	high earners hide income
Structural	the value cannot exist	"months since last delinquency" when there was none

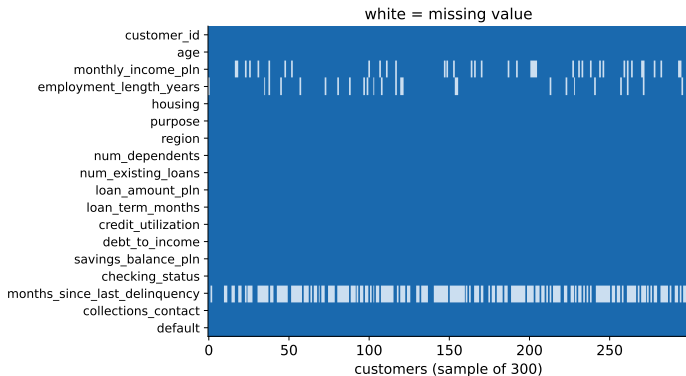
The **mechanism** decides whether it is safe to drop, impute, or *model*.

Our portfolio: how much is missing, and why



- Three columns, three different mechanisms - and three different correct treatments.
- First step is always the same: **count and name** the missingness before touching it.

Seeing the pattern of missingness



- A missingness matrix shows *structure*: which columns, how often, and whether gaps co-occur in the same rows.
- In the lab: the same view via `df.isna()` - no extra library needed.

Option 1: deletion - when dropping is (not) safe

Dropping rows (listwise deletion)

- Safe-ish only under **MCAR** - you lose power, but no bias.
- Under MAR/MNAR it **biases the sample**: dropping income-missing rows here removes high earners.

Dropping columns

- Defensible when a column is mostly empty *and* carries no signal - check both.
- Here, months_since_last_delinquency is ~64% missing and still one of the **most informative** columns in the data.

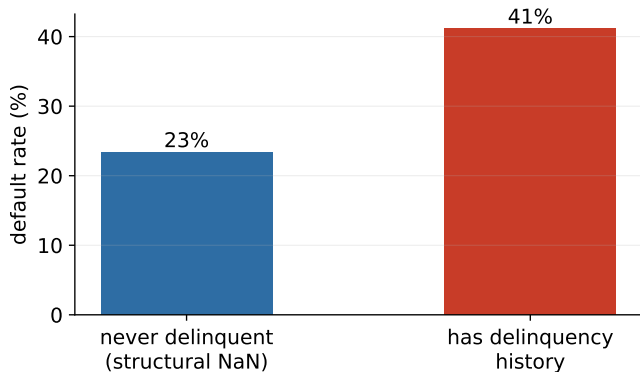
Deletion is a modelling decision, not housekeeping - justify it like one.

Option 2: the imputation menu

Method	Good	Risky
Mean / median	simple, fast; median survives skew	shrinks variance; distorts under MAR/MNAR
Mode (categorical)	simple	inflates the majority level
Group-wise median	respects segments (e.g. per region)	groups must be big enough
Model-based / KNN	uses relationships between features	complexity, leakage risk if fit on all data
Constant + indicator	keeps “was missing” as signal	adds columns

Golden rule: fit any imputation on the **training data only** - a preview of Block 3's leakage discussion.

A missing value can be a signal



- Customers with the structural NaN (never delinquent) default far less often - the **gap itself predicts**.
- Any treatment that erases the gap erases the signal.

The punchline of Block 1

A missing value can be the strongest signal you have

Customers with *no prior delinquency* carry a structural NaN in “months since last delinquency”. Mean-imputing it would **invent a delinquency history that never happened** - and erase a real signal.

- Honest first move: add a **missing-indicator**, *then* impute the number.
- In regulated lending, the reason you give a customer must be truthful. Imputation choices are not cosmetic - they change the story.

A first warning about leakage

Leakage = using information that will not exist at prediction time

Our dataset contains `collections_contact` - whether collections called the customer. It is **recorded after default happens**. A model using it looks brilliant in the notebook and is worthless in production.

The time-travel test (ask for every feature):

“Would I have known this value at the moment of the decision?”

- If the answer is no - the feature is a time traveller. Drop it.
- Full treatment (and a live demonstration) in Block 3.

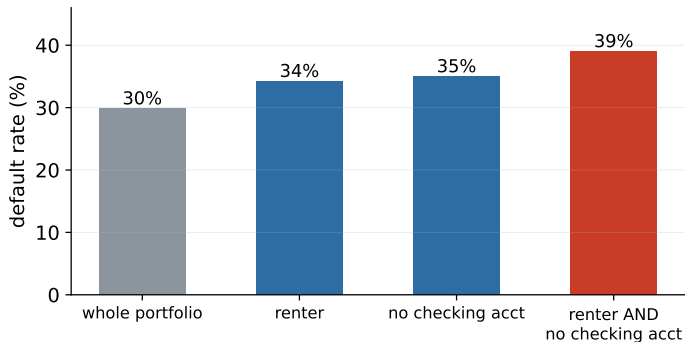
Feature engineering: where domain knowledge becomes signal

New columns that expose structure the model would struggle to find alone. The missing-indicator you just built *is* feature engineering - here is the wider toolbox:

Move	Recipe	Portfolio example
Ratio	divide two raw columns	instalment / income
Flag	boolean condition	has_delinquency_history
Binning	continuous → bands	utilisation: low / mid / maxed-out
Interaction	combine two features	renter <i>and</i> no checking account
Date arithmetic	differences, tenures	months since last delinquency

Look at the data dictionary again: debt_to_income is a ratio - **someone engineered it before us.**

Worked example: combining two weak flags



- Renting alone and no-checking-account alone are each mildly risky; the **interaction flag** isolates the riskiest segment (39%, $n = 318$ - large enough to trust).
- And recall the strongest engineered feature so far: the `delinq_missing` **indicator** ($|r| \approx 0.19$ with default - more than any raw numeric column).

Feature engineering: rules of the game

- ➊ **Application-time information only** - every engineered feature must pass the **time-travel test**, same as raw ones.
- ➋ **Fit transforms on training data only** - means for scaling, bin edges, encodings (the leakage-safe pipeline, Block 3).
- ➌ **Simple and explainable beats clever** - in regulated lending every feature may end up in an adverse-action reason.
- ➍ **Document the recipe** - production must compute the feature *identically*, or the model silently degrades.
- ➎ **Expect diminishing returns** - five features built from domain sense beat five hundred generated blindly.

Hands-on: exercise 5 in today's lab asks you to engineer one feature from loan amount, term, and income - and defend it.

Encodings: making categories digestible (preview)

Most models eat numbers, not labels. The scale of measurement picks the encoding:

Encoding	Use for	Watch out for
One-hot	nominal (purpose, region)	column explosion with many levels
Ordinal codes	truly ordered (checking status)	only when the order is real
Target encoding	high-cardinality nominal	leakage - fit on train only
WoE (credit tradition)	scorecard features	needs binning first; Block 2

- Encoding nominal as integers invents an order; one-hot on ordinal discards one. *Scale first, encoding second.*
- Full treatment next week, when regression needs numeric inputs.

Reproducibility from day one

- **Fix your random seeds** - `random_state=42` everywhere a coin is flipped (splits, models, samples).
- **Pin your environment** - `uv lockfile`; “works on my machine” is not a result.
- **One-command rule** - a colleague should reproduce your numbers by running one thing, top to bottom.
- **Data provenance** - record where the data came from and when; datasets change under your feet.

If it cannot be reproduced, it is an **anecdote**, not an analysis - and the project rubric scores it.

Notebooks: superpower and trap

Superpower

- code + plots + narrative in one place
- perfect for EDA and for the project report

Trap: hidden state

- cells run out of order $\Rightarrow$ results that cannot be recreated
- deleted cells leave live variables behind

The habit that saves you

Before trusting any result: **Restart kernel & run all**. If it breaks, it was already broken - you just didn't know.

Version control - yes, for analysts too

- **Commit small and often** - each commit one logical step; messages that say *why*.
- **What goes in**: code, notebooks, configs, figure scripts.
- **What stays out**: raw data (use loaders), credentials, .env files, 100 MB artefacts.
- **Why bother in a student project?** You will want yesterday's working version at 23:50 before the deadline.

`git init` today costs 10 seconds. Reconstructing a deleted analysis costs a weekend.

Reading errors & asking good questions

When code explodes

- Read the traceback **bottom-up**: the last line is the error, the lines above are where it happened.
- Reproduce it in the smallest possible example - half the time the bug evaporates as you shrink it.

When you ask for help (colleague, forum, AI assistant)

- say what you expected, what happened instead, and what you tried
- include the minimal example - not a screenshot of 400 lines
- the discipline of writing the question well often *answers* it

Communicating results to humans

- **Lead with the decision**, not the method: “we can cut losses 12% at the same approval rate” beats “our AUC is 0.79”.
- **State uncertainty honestly** - ranges and caveats build trust; false precision destroys it.
- **One chart, one message** - if a slide needs a paragraph of explanation, it is two slides.
- **Know your audience** - the risk committee, the regulator, and your fellow data scientist need three different stories.

Your model's impact is capped by your ability to **explain** it.

Beginner traps (we will meet them all)

- 1 **Evaluating on training data** - your score is a fantasy (Block 2).
- 2 **Metric worship** - optimising accuracy on a 97/3 problem (Block 3).
- 3 **Complexity first** - neural network before a logistic baseline; you can't tell if the extra complexity earned anything (Blocks 2, 6).
- 4 **Not looking at the data** - today's whole point.
- 5 **Torturing the data until it confesses** - test 100 hypotheses, report the 5 that "worked".
- 6 **Silently deleting inconvenient rows** - outliers and missing values carry signal.

The course subtitle is a promise: **how not to fool yourself**.

Laboratory

Notebook 01_eda_missingness - laptops out.

In the lab today

- 1 **Load** the portfolio and run the first-contact commands.
 - 2 **Profile** the target and the key numeric / categorical features (the plots from today, hands-on).
 - 3 **Hunt** the missingness: count it, plot it, name the mechanism for each column.
 - 4 **Experiment**: mean-impute vs indicator-plus-impute - watch what happens to the delinquency signal.
 - Runs locally or on Colab; data loads itself. Light on Python? Start with 00_python_primer.
- “Done” = you can defend, in one paragraph, how you would treat each missing column and why.

Summary

What to take home from Block 1.

Block 1 in five sentences

- 1 Data mining starts from a **decision**, and translating it into a learning task is your job, not the algorithm's.
- 2 We focus on **binary classification** and **regression**, with the rest of the map mentioned as we pass it.
- 3 **EDA before models**: target balance, distributions, bivariate relationships, and always plot (Anscombe!).
- 4 Missingness has a **mechanism** (MCAR / MAR / MNAR / structural) - and the mechanism dictates the treatment.
- 5 A missing value can be **signal**: indicator first, impute second, and never invent a history that didn't happen.

Project reminder

- Before next class (Block 2, 13.10): teams of 3, formed and registered. That is this week's only project task.
- Looking one week further: topic + short description due **before Block 3** - if no idea comes, consult early.

Full brief, timeline, grading and the AI rule: the project slides at the start of today's deck.

Exam practice - multiple choice

Q1. `months_since_last_delinquency` is NaN exactly for customers who were *never* delinquent. The missingness mechanism is:

- ☐ a) MCAR - missing completely at random
- ☐ b) MAR - explained by another observed column
- ☐ c) MNAR - depends on the unobserved value itself
- ☐ d) structural - the value cannot exist

Q2. A default model reports 92% accuracy on a portfolio with an 8% default rate. The honest reading:

- ☐ a) excellent - deploy it
- ☐ b) says little: predicting "repaid" for everyone also scores 92%
- ☐ c) the model is overfit
- ☐ d) accuracy is too low for credit work

Exam practice - open question

In the style of the term-0 exam (~60% open questions)

A bank wants a default model for *all future applicants*, but its dataset contains only loans it previously *approved*. Explain the problem this creates for the model, name the phenomenon, and propose one mitigation.

Write 4-6 sentences. Naming the concept is a third of the credit; explaining the consequence is the rest.

Next week: regression, re-framed.

From *inference* to *prediction*.

Keep learning - Block 1 reading

- **Shearer** - *The CRISP-DM Model: The New Blueprint for Data Mining* (J. of Data Warehousing, 2000) - the process spine of this course, from the source.
- **Anscombe** - *Graphs in Statistical Analysis* (The American Statistician, 1973) - today's quartet, in the author's own words.
- **Rubin** - *Inference and Missing Data* (Biometrika, 1976) - where MCAR / MAR / MNAR actually come from.
- **Wickham** - *Tidy Data* (J. of Statistical Software, 2014) - how to shape a table before any analysis touches it.

Course-wide references (ISLR, Géron, the scikit-learn User Guide) live in the repo README.